

MICROSERVICES – 50 Deep Interview Questions (3-Line Answers)

1. Why choose Microservices over Monolith?

Microservices allow independent deployment of services.

Each service can scale independently based on load.

It improves fault isolation and team autonomy.

2. What are the main characteristics of Microservices?

Services are loosely coupled and independently deployable.

Each service owns its database.

Communication happens via lightweight protocols like HTTP or messaging.

3. What is Service Discovery?

Service Discovery allows services to register themselves dynamically.

Other services discover them without hardcoded URLs.

Tools like Eureka or Consul are commonly used.

4. What is API Gateway?

API Gateway acts as a single entry point for all client requests.

It handles routing, authentication, and rate limiting.

Reduces direct exposure of internal services.

5. What is Circuit Breaker Pattern?

Circuit breaker stops calls to failing services.

Prevents cascading failures across the system.

Resilience4j is commonly used for this.

6. What is Bulkhead Pattern?

Bulkhead isolates failures in specific components.

Prevents entire system from crashing.

Improves system resilience.

7. What is Saga Pattern?

Saga manages distributed transactions across services.

It uses either choreography or orchestration approach.

Ensures eventual consistency.

8. What is Event-Driven Architecture?

Services communicate via events instead of direct calls.

Improves decoupling and scalability.

Kafka or RabbitMQ are commonly used.

9. What is Idempotency?

Idempotent operations produce same result when repeated.

Important in payment and retry scenarios.

Prevents duplicate transactions.

10. What is Database per Service pattern?

Each microservice owns its own database.

Prevents tight coupling between services.

Ensures data autonomy.

11. How do Microservices communicate?

Using REST APIs or messaging systems.

Synchronous communication uses HTTP.

Asynchronous uses message brokers.

12. What is Centralized Logging?

Logs from all services are collected in one system.

ELK stack is commonly used.

Helps debug distributed systems.

13. What is Distributed Tracing?

Tracks request flow across multiple services.

Helps identify latency bottlenecks.

Tools like Zipkin or Jaeger are used.

14. What is Load Balancing?

Distributes traffic across multiple instances.

Improves availability and scalability.

Can be client-side or server-side.

15. What is Horizontal Scaling?

Adding more instances of a service.

Improves performance under heavy load.

Common in cloud environments.

16. What is Vertical Scaling?

Increasing resources of a single instance.

Limited by hardware capacity.

Less flexible compared to horizontal scaling.

17. What is CAP Theorem?

States Consistency, Availability, Partition tolerance trade-off.

Distributed systems can guarantee only two at a time.

Microservices often choose AP.

18. What is Eventual Consistency?

Data becomes consistent over time.

Used in distributed systems.

Common in Saga pattern.

19. What is Choreography Saga?

Services react to events independently.

No central coordinator.

Simple but hard to track in complex systems.

20. What is Orchestration Saga?

Central service coordinates transaction steps.

Easier to manage complex workflows.

Introduces orchestration dependency.

21. What is Containerization?

Packaging application with dependencies.

Ensures consistent deployment.

Docker is commonly used.

22. What is Kubernetes?

Container orchestration platform.

Manages deployment, scaling, and networking.

Provides self-healing features.

23. What is Blue-Green Deployment?

Two identical environments exist.

Switch traffic after validation.

Reduces downtime.

24. What is Canary Deployment?

New version deployed to small subset of users.

Gradually increases traffic.

Reduces deployment risk.

25. What is API Versioning?

Maintains backward compatibility.

Handled via URL or headers.

Prevents breaking existing clients.

26. What is Rate Limiting?

Limits number of requests per client.

Prevents abuse and overload.

Implemented in API Gateway.

27. What is Authentication in Microservices?

Handled using JWT or OAuth2.

Token validated in each service.

Improves security in distributed systems.

28. What is Authorization?

Controls access to resources.

Based on roles or permissions.

Integrated with security frameworks.

29. What is Service Mesh?

Handles service-to-service communication.

Provides traffic control and security.

Example: Istio.

30. What is Observability?

Combination of logging, metrics, and tracing.

Helps monitor system behavior.

Critical in distributed systems.

31. What is Backward Compatibility?

Ensures new changes do not break existing APIs.

Handled via versioning.

Important for client stability.

32. What is Retry Mechanism?

Retries failed requests automatically.

Improves resilience.

Should use exponential backoff.

33. What is Timeout?

Defines maximum wait time for response.

Prevents resource blocking.

Used with circuit breaker.

34. What is Fail Fast?

System immediately fails when dependency is down.

Prevents resource exhaustion.

Used with circuit breakers.

35. What is Data Replication?

Copies data across nodes.

Improves availability.

Used in distributed databases.

36. What is CQRS?

Separates read and write models.

Improves scalability.

Often combined with event sourcing.

37. What is Event Sourcing?

Stores state changes as events.

Allows system state reconstruction.

Improves auditability.

38. What is Zero Downtime Deployment?

Deploy without stopping system.

Achieved using rolling updates.

Important in production systems.

39. What is Health Check?

Monitors service availability.

Used by load balancer.

Implemented via Actuator.

40. What is Self-Healing?

System automatically replaces failed instances.

Provided by Kubernetes.

Improves reliability.

41. What is Data Consistency Challenge?

Multiple services update different databases.

Hard to maintain strong consistency.

Solved using Saga pattern.

42. What is Service Contract?

Defines API interface between services.

Ensures clear communication.

Maintained via OpenAPI/Swagger.

43. What is Sidecar Pattern?

Helper container deployed alongside main service.

Handles logging or monitoring.

Common in service mesh.

44. What is Centralized Configuration?

Stores config in single server.

Avoids duplication across services.

Example: Spring Cloud Config.

45. What is Rolling Update?

Gradually replaces old instances with new ones.

No downtime required.

Used in Kubernetes.

46. What is Message Broker?

Handles asynchronous communication.

Examples include Kafka and RabbitMQ.

Improves decoupling.

47. What is Dead Letter Queue?

Stores failed messages.

Prevents message loss.

Used in messaging systems.

48. What is API Composition?

Combines responses from multiple services.

Handled by API Gateway.

Reduces client complexity.

49. What is Latency Issue?

Delay in response between services.

Caused by network overhead.

Solved using caching and optimization.

50. What is Scalability Challenge in Microservices?

Network overhead increases with more services.

Monitoring becomes complex.

Requires proper orchestration and observability.